



Projeto II — API REST com Banco de Dados MySQL

Esta atividade foi desenvolvida como parte da avaliação da disciplina de Programação Web do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de São Paulo, Câmpus Boituva. Se houver dúvidas sobre o enunciado, entre em contato através do e-mail: anisio.silva@ifsp.edu.br

1 Objetivo e Contextualização

O objetivo deste projeto é evoluir a API REST para gestão de uma concessionária de veículos desenvolvida no **Projeto I**. Nesta segunda etapa, a arquitetura MVC será integrada de forma definitiva a um banco de dados relacional **MySQL**, eliminando por completo o armazenamento volátil baseado em estruturas de dados em memória (*arrays*).

As regras de negócio originais permanecem rigorosamente as mesmas. Contudo, nesta nova especificação, as rotas foram unificadas em um único arquivo centralizador (**router.ts**) e os contratos de resposta (**Responses**) foram formalmente definidos e padronizados para garantir a consistência e previsibilidade da API.

2 Estrutura de Pastas e Arquitetura

O projeto deve seguir estritamente a estrutura modular apresentada em aula, isolando a conexão com o banco e centralizando as rotas da aplicação:

Listing 1: Estrutura de arquivos do Projeto II

```
projeto_II/  
|- src/  
|   |- app.ts  
|   |- database/  
|       |- mysql.ts           # Pool de conexao com o MySQL (mysql2)  
|   |- routes/  
|       |- router.ts          # Definicao de todas as rotas  
|   |- model/  
|       |- Cliente.ts  
|       |- Vendedor.ts  
|       |- Carro.ts  
|       |- Estoque.ts  
|       |- NotaFiscal.ts  
|   |- repository/  
|       |- ClienteRepository.ts # Instrucoes SQL  
|       |- VendedorRepository.ts  
|       |- CarroRepository.ts  
|       |- EstoqueRepository.ts
```

```

|   |   |- NotaFiscalRepository.ts
|   |- service/
|   |   |- ClienteService.ts
|   |   |- VendedorService.ts
|   |   |- CarroService.ts
|   |   |- EstoqueService.ts
|   |   |- NotaFiscalService.ts
|   |- controller/
|   |   |- ClienteController.ts
|   |   |- VendedorController.ts
|   |   |- CarroController.ts
|   |   |- EstoqueController.ts
|   |   |- NotaFiscalController.ts
|- node_modules/
|- tsconfig.json
|- package.json

```

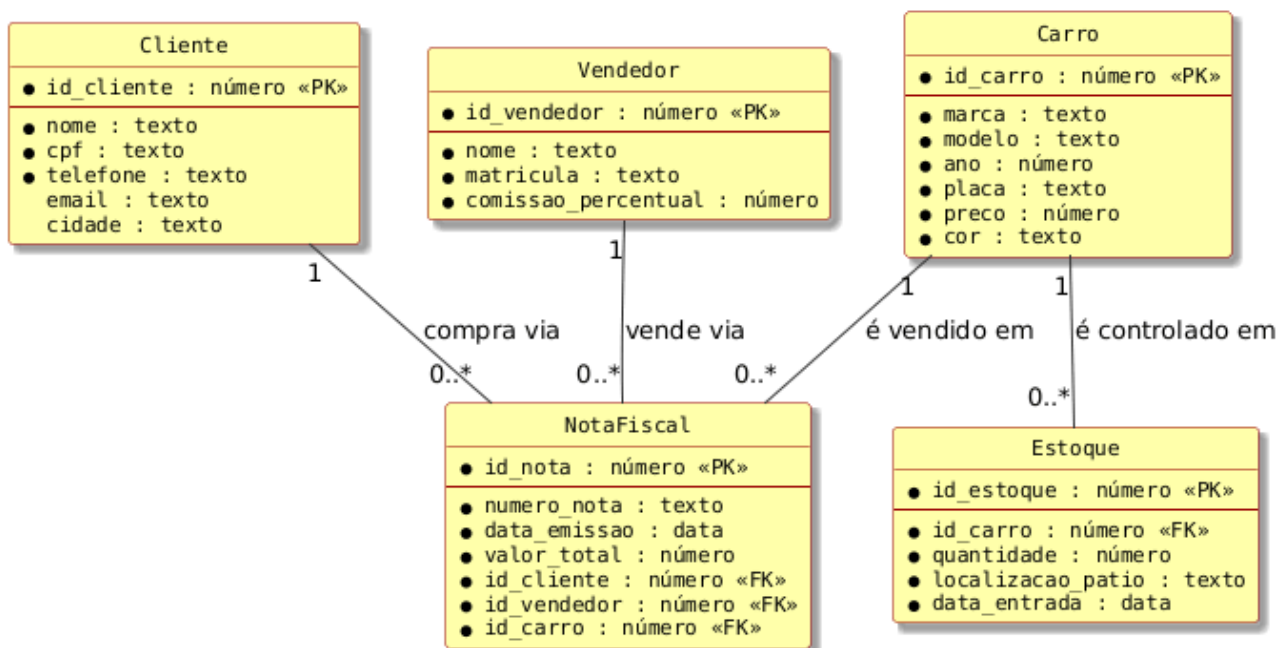


Figura 1: Diagrama de Entidade-Relacionamento do sistema

3 Regras de Negócio Detalhadas

As regras de negócio a seguir regulam o funcionamento do sistema. Todas as validações devem ser implementadas na camada de **serviço**, retornando respostas HTTP adequadas em caso de violação.

RN01 — Cadastro de Clientes

- O campo **cpf** é obrigatório e deve ser único no sistema. Não é permitido cadastrar dois clientes com o mesmo CPF.

- Os campos **nome**, **cpf** e **telefone** são obrigatórios. Os campos **email** e **cidade** são opcionais.
- Não é permitido remover um cliente que possua notas fiscais vinculadas a ele.

RN02 — Cadastro de Vendedores

- O campo **matricula** é obrigatório e deve ser único no sistema.
- O campo **comissao_percentual** deve ser um número positivo entre **0** e **30** (por cento).
- Não é permitido remover um vendedor que possua notas fiscais vinculadas a ele.

RN03 — Cadastro de Carros

- O campo **placa** é obrigatório e deve ser único no sistema. Não é permitido cadastrar dois carros com a mesma placa.
- O campo **ano** deve ser um número inteiro entre **1950** e o ano atual mais 1 (admitindo carros do próximo ano já em estoque).
- O campo **preco** deve ser um valor positivo maior que zero.
- Não é permitido remover um carro que possua registros em estoque ou notas fiscais vinculadas.

RN04 — Controle de Estoque

- Cada registro de estoque está vinculado a exatamente **um** carro (via **id_carro**). O carro referenciado deve existir no sistema.
- O campo **quantidade** deve ser um inteiro maior ou igual a **zero**. Uma quantidade igual a zero significa que o item está indisponível mas ainda consta no histórico.
- O campo **data_entrada** não pode ser uma data futura em relação à data atual do servidor.
- Não pode existir mais de um registro de estoque ativo para o mesmo **id_carro**. Para atualizar a quantidade, deve-se usar o endpoint de atualização (PUT).

RN05 — Emissão de Nota Fiscal

- Uma nota fiscal somente pode ser emitida se o carro associado possuir **quantidade** > 0 em estoque. Ao emitir, a quantidade em estoque é automaticamente decrementada em **1** unidade.
- O campo **numero_nota** é obrigatório e deve ser único no sistema.
- O campo **valor_total** deve ser positivo e maior que zero.
- Os campos **id_cliente**, **id_vendedor** e **id_carro** são obrigatórios e devem referenciar registros existentes no sistema.
- A **data_emissao** não pode ser uma data futura em relação à data atual do servidor.
- Notas fiscais **não podem ser removidas** após a emissão, garantindo integridade histórica.

RN06 — Consultas e Listagens

- Deve ser possível listar todas as notas fiscais de um cliente específico.
- Deve ser possível listar todas as notas fiscais de um vendedor específico.
- Deve ser possível consultar o estoque atual de um carro pelo seu id.
- Deve ser possível listar todos os carros com estoque disponível (quantidade > 0).

4 Documentação da API

A API segue os princípios REST, utilizando os verbos HTTP GET, POST, PUT e DELETE. Todas as requisições e respostas utilizam o formato **JSON**. A URL base deve ser: `http://localhost:3000`.

4.1 Clientes — /clientes

Método	Rota	Descrição	Response
GET	/clientes	Lista todos os clientes cadastrados.	[{cliente}]
GET	/clientes/:id	Retorna os dados de um cliente pelo id.	{cliente}
POST	/clientes	Cadastra um novo cliente.	{cliente}
PUT	/clientes/:id	Atualiza os dados de um cliente existente.	{cliente}
DELETE	/clientes/:id	Remove um cliente (somente se não possuir notas fiscais).	{cliente}
GET	/clientes/notas/:id	Lista todas as notas fiscais de um cliente.	[{nota}]

Corpo da requisição — POST / PUT /clientes

```
{  
  "nome":      "João da Silva",  
  "cpf":       "123.456.789-00",  
  "telefone":  "(11) 99999-0000",  
  "email":     "joao@email.com",  
  "cidade":    "São Paulo"  
}
```

Response — 200 / 201

```
{  
  "id_cliente": 1,  
  "nome":       "João da Silva",  
  "cpf":        "123.456.789-00",  
  "telefone":   "(11) 99999-0000",  
  "email":      "joao@email.com",  
}
```

```

        "cidade":      "São Paulo"
    }

```

4.2 Vendedores — /vendedores

Método	Rota	Descrição	Response
GET	/vendedores	Lista todos os vendedores.	[{vendedor}]
GET	/vendedores/:id	Retorna um vendedor pelo id.	{vendedor}
POST	/vendedores	Cadastra um novo vendedor.	{vendedor}
PUT	/vendedores/:id	Atualiza um vendedor existente.	{vendedor}
DELETE	/vendedores/:id	Remove um vendedor (sem notas vinculadas).	{vendedor}
GET	/vendedores/notas/:id	Lista todas as notas fiscais de um vendedor.	[{nota}]

Corpo da requisição — POST / PUT /vendedores

```

{
    "nome":              "Ana Souza",
    "matricula":         "VND-001",
    "comissao_percentual": 5.5
}

```

Response — 200 / 201

```

{
    "id_vendedor":      2,
    "nome":              "Ana Souza",
    "matricula":         "VND-001",
    "comissao_percentual": 5.5
}

```

4.3 Carros — /carros

Método	Rota	Descrição	Response
GET	/carros	Lista todos os carros.	[{carro}]
GET	/carros/:id	Retorna um carro pelo id.	{carro}
GET	/carros/disponiveis	Lista carros com estoque > 0.	[{carro}]
POST	/carros	Cadastra um novo carro.	{carro}

Método	Rota	Descrição	Response
PUT	/carros/:id	Atualiza um carro existente.	{carro}
DELETE	/carros/:id	Remove um carro (sem estoque ou notas).	{carro}

Corpo da requisição — POST / PUT /carros

```
{
  "marca": "Toyota",
  "modelo": "Corolla",
  "ano": 2024,
  "placa": "ABC-1234",
  "preco": 110000.00,
  "cor": "Prata"
}
```

Response — 200 / 201

```
{
  "id_carro": 3,
  "marca": "Toyota",
  "modelo": "Corolla",
  "ano": 2024,
  "placa": "ABC-1234",
  "preco": 110000.00,
  "cor": "Prata"
}
```

4.4 Estoque — /estoque

Método	Rota	Descrição	Response
GET	/estoque	Lista todos os registros de estoque.	[{estoque}]
GET	/estoque/:id	Retorna um registro de estoque pelo id.	{estoque}
GET	/estoque/carro/:id_carro	Retorna o estoque de um carro específico.	{estoque}
POST	/estoque	Cria um novo registro de estoque.	{estoque}
PUT	/estoque/:id	Atualiza quantidade ou localização.	{estoque}
DELETE	/estoque/:id	Remove um registro de estoque.	{estoque}

Corpo da requisição — POST /estoque

```
{
    "id_carro": 1,
    "quantidade": 5,
    "localizacao_patio": "Galpão A-3",
    "data_entrada": "2025-06-01"
}
```

Response — 200 / 201

```
{
    "id_estoque": 4,
    "id_carro": 1,
    "quantidade": 5,
    "localizacao_patio": "Galpão A-3",
    "data_entrada": "2025-06-01"
}
```

4.5 Notas Fiscais — /notas

Método	Rota	Descrição	Response
GET	/notas	Lista todas as notas fiscais.	[{nota}]
GET	/notas/:id	Retorna uma nota fiscal pelo id.	{nota}
POST	/notas	Emite uma nova nota fiscal (decrementa estoque).	{nota}

Corpo da requisição — POST /notas

```
{
    "numero_nota": "NF-2025-0001",
    "data_emissao": "2025-06-10",
    "valor_total": 110000.00,
    "id_cliente": 1,
    "id_vendedor": 2,
    "id_carro": 3
}
```

Response — 201

```
{
    "id_nota": 5,
    "numero_nota": "NF-2025-0001",
    "data_emissao": "2025-06-10",
    "valor_total": 110000.00,
    "id_cliente": 1,
    "id_vendedor": 2,
    "id_carro": 3
}
```

4.6 Padrão de Respostas HTTP

A tabela a seguir descreve os códigos de status esperados em cada situação:

Status	Situação
200 OK	Requisição bem-sucedida (GET, PUT).
201 Created	Recurso criado com sucesso (POST).
400 Bad Request	Dados inválidos, campos obrigatórios ausentes ou violação de regra de negócio (ex.: CPF duplicado, carro sem estoque).
404 Not Found	Recurso não encontrado pelo <code>id</code> informado.
409 Conflict	Violação de unicidade (CPF, matrícula, placa, número da nota já existentes).
422 Unprocessable	Regra de negócio impede a operação (ex.: remoção de cliente com notas, estoque insuficiente).

5 Entrega e Avaliação

Critérios de Avaliação

- Implementação correta da arquitetura MVC
- Cumprimento de todos os requisitos funcionais
- Aplicação rigorosa das regras de negócio
- Qualidade e organização do código
- Documentação adequada
- Desempenho na arguição

Entrega

- Data limite:** 29/06/2026 até às 6:00 AM
- Plataforma:** Moodle
- Submeter o **link do repositório público no GitHub** com o projeto completo contendo a *collection* do Postman com todos os testes funcionais e histórico de commits

Critérios de Avaliação

Critério	Peso	O que será observado
Organização dos commits	10%	Histórico contínuo e mensagens descritivas. Commits pontuais a cada funcionalidade implementada, sem envio único no final.

Critério	Peso	O que será observado
Organização do código (MVC)	10%	Separação correta entre <i>model</i> , <i>repository</i> , <i>service</i> e <i>controller</i> . Cada camada com responsabilidade única e bem definida.
Funcionalidade	35%	Todos os endpoints respondendo corretamente, status HTTP adequados e regras de negócio aplicadas. A <i>collection</i> do Postman deve cobrir todos os endpoints com ao menos um exemplo de requisição válida e um de erro.
Lógica de programação	30%	Clareza e coerência do código: uso correto de estruturas de repetição, condicionais, funções e tipos TypeScript.
Arguição	15%	Capacidade de explicar as decisões técnicas e responder a questionamentos sobre o próprio código.
Total	100%	

Arguição

- **Data:** 03/06/2026
- **Requisitos:**
 - Demonstração do sistema funcionando
 - Explicação das decisões técnicas
 - Resposta a questionamentos

Observações

- Ausência na arguição resultará em **nota zero**, independentemente da entrega.
- Repositórios sem histórico de commits (projeto enviado de uma só vez) terão o critério correspondente zerado.
- O código entregue deve ser **de autoria própria**. Trabalhos idênticos ou gerados integralmente por ferramentas de IA serão desconsiderados.

Observação: Ausência em qualquer etapa resultará em nota zero.

Testador de Endpoints (Opcional)

Para ajudar no desenvolvimento e na validação dos endpoints da sua API, disponibilizamos um script de testes automatizados.

Importante

O uso deste testador é **100% opcional** e serve apenas como uma ferramenta de auxílio para você verificar se as suas rotas estão a funcionar como esperado. A ausência do uso deste script não terá qualquer impacto na nota, desde que a aplicação cumpra todos os requisitos solicitados no enunciado.

Como baixar e executar o testador:

1. Download do arquivo:

Faça o download do arquivo de testes através do seguinte link:

Clique aqui para baixar o `tester_api.mjs`.

Guarde o arquivo na pasta do seu projeto ou num local da sua preferência.

2. Inicie a sua aplicação:

Certifique-se de que a sua aplicação Web está ativa e a rodar localmente na porta 3000. O testador fará as requisições diretamente para:

`http://localhost:3000`

3. Execute o testador:

Abra um **segundo terminal** (mantenha o terminal da sua aplicação em execução), navegue até à pasta onde guardou o arquivo baixado e execute o seguinte comando utilizando o Node.js:

```
node tester_api.mjs
```

4. Analise os resultados:

O script exibirá no terminal o retorno de cada teste, indicando quais endpoints responderam corretamente e quais precisam de ajustes.